steiitb

Tugas Besar 1 IF3070 Dasar Inteligensi Artifisial 2026/2027

Pengembangan ***Cargo Optimizer*** untuk Optimasi Penempatan Kendaraan pada **Geladak Kapal**



Proposed by



18224026
M.B. A. Narandita



18224040
Derren Christopher B.



18224044
Rafeyfa Dea Putri. R

ABSTRAK

Laporan ini membahas penerapan algoritma *Hill Climbing* pada masalah penempatan kendaraan di geladak kapal kargo yang bertujuan memaksimalkan total *ShippingFee*. Geladak dimodelkan sebagai grid 2D berukuran W × L dengan batas berat maksimum, sedangkan setiap kendaraan memiliki dimensi, orientasi, tarif, berat, dan *ETA*. Solusi berupa pilihan kendaraan yang diangkut beserta posisi dan orientasinya, dengan tiga batasan yaitu tidak tumpang tindih, tidak melewati batas geladak, dan tidak melebihi kapasitas berat.

Steepest Ascent dan *Hill Climbing* dengan *Sideways Move* memiliki inti algoritma yang sama. Pencarian dimulai dari state acak yang selalu valid. Dari situ, tetangga dibuat lewat tiga jenis perubahan (*swap, move, rotate*), dan nilainya dihitung hanya dari bagian yang berubah supaya program berjalan cepat. Fungsi objektif kita tidak peduli posisi objek, jadi banyak state bertetangga punya nilai yang sama (area datar). Karena itu, yang paling menarik dianalisis adalah bagaimana kedua varian menangani area datar tersebut.

Kata kunci: *local search*, *hill climbing*, *steepest ascent*, *sideways move*, penempatan kendaraan

DAFTAR ISI

ABSTRAK **2**
DAFTAR ISI **3**
DAFTAR TABEL **5**
BAB I **6**
DESKRIPSI PERSOALAN **6**
1.1 Latar Belakang Persoalan 6
1.2 Definisi dan Batasan Persoalan 6
1.3 Tujuan 7
BAB II **8**
IMPLEMENTASI ALGORITMA **8**
2.1 Representasi State 8
2.2 Fungsi Objektif dan Constraint 8
2.3 Fungsi Successor dan Pergerakan Tetangga 9
2.4 Implementasi Varian Hill Climbing 9
2.4.1 Steepest Ascent Hill Climbing 10
2.4.2 Hill Climbing dengan Sideways Move 11
2.5 Implementasi Simulated Annealing 13
2.6 Implementasi Genetic Algorithm 16
BAB III **20**
HASIL PENGUJIAN DAN ANALISIS **20**
3.1 Konfigurasi Eksperimen 20
3.2 Hasil Eksperimen 21
3.2.1 Steepest Ascent Hill Climbing 21
3.2.2 Hill Climbing dengan Sideways Move 21
3.2.3 Simulated Annealing 22
3.2.4 Genetic Algorithm 23
3.3 Analisis Hasil Eksperimen 28
BAB IV **32**
KESIMPULAN DAN SARAN **32**
4.1 Kesimpulan 32
4.2 Saran 32
PEMBAGIAN TUGAS **34**
DAFTAR PUSTAKA **35**

DAFTAR TABEL

Tabel 2.1 Struktur Data Utama.....7
Tabel 2.2 Jenis Perubahan..... 8
Tabel 2.3 Implementasi Hill Climbing dengan Steepest Ascent..... 9
Tabel 2.4 Implementasi Hill Climbing dengan Sideways Move.....11
Tabel 2.5 Implementasi Simulated Annealing.....13
Tabel 2.6 Implementasi Genetic Algorithm.....15
Tabel 3. 1 Konfigurasi eksperimen..... 19
Tabel 3.2 Hasil Eksperimen Steepest Ascent Hill Climbing.....20
Tabel 3.3 Hasil Eksperimen Hill Climbing dengan Sideways Move.....20
Tabel 3.4 Hasil Eksperimen Simulated Annealing..... 21
Tabel 3.5 Hasil Eksperimen Genetic Algorithm (Populasi 50 dan iterasi 20)..... 22
Tabel 3.6 Hasil Eksperimen Genetic Algorithm (Populasi 50 dan iterasi 100)..... 23
Tabel 3.7 Hasil Eksperimen Genetic Algorithm (Populasi 50 dan iterasi 300).....24
Tabel 3.8 Hasil Eksperimen Genetic Algorithm (Populasi 10 dan Iterasi 100)..... 25
Tabel 3.9 Hasil Eksperimen Genetic Algorithm (Populasi 150 dan Iterasi 100).....26
Tabel 3.10 Hasil Eksperimen Genetic Algorithm (Populasi 70 dan Iterasi 100)..... 27
Tabel 3.11 Analisis Hasil Pertanyaan 1.....27
Tabel 3.12 Analisis Hasil Pertanyaan 2..... 28
Tabel 3.13 Analisis Hasil Pertanyaan 3..... 29
Tabel 3.14 Analisis Hasil Pertanyaan 4.....29
Tabel 3.15 Analisis Hasil Pertanyaan 5..... 29
Tabel 3.16 Analisis Hasil Pertanyaan 6..... 30

BAB I
DESKRIPSI PERSOALAN

1.1 Latar Belakang Persoalan

Pengangkutan kendaraan menggunakan kapal membutuhkan penataan yang efisien. Geladak kapal memiliki ruang terbatas dan kapasitas berat maksimum, sedangkan kendaraan yang ingin diangkut memiliki ukuran, berat, dan tarif pengiriman yang berbeda-beda. Tidak semua kendaraan dapat diangkut sekaligus, sehingga perlu dipilih kendaraan mana yang diangkut serta bagaimana posisi dan orientasinya di geladak agar keuntungan total maksimal.

Masalah ini merupakan gabungan dari dua masalah optimasi kombinatorial, yaitu *knapsack* (memilih kendaraan dengan batas berat) dan *2D bin packing* (menyusun kendaraan berbentuk persegi panjang di bidang terbatas tanpa tumpang tindih). Ruang pencarian yang dihasilkan sangat besar sehingga pencarian menyeluruh tidak praktis. Karena itu digunakan pendekatan *local search*, yaitu algoritma yang bergerak dari satu *state* ke *state* tetangganya untuk memperbaiki nilai objektif.

Persoalan serupa juga bisa ditemukan dalam kasus operasional pelabuhan dan logistik, misalnya pemuatan kendaraan ke kapal pengangkut atau penataan muatan pada ruang kargo. Pengambil keputusan harus menyeimbangkan tiga hal sekaligus: keuntungan dari tiap kendaraan, ruang geladak yang terbatas, dan batas berat yang aman. Penyusunan secara manual cenderung menyisakan ruang kosong yang terpecah-pecah, sehingga kendaraan bernilai tinggi tidak dapat diangkut padahal total luas geladak yang tersisa sebenarnya masih cukup.

Sebagai gambaran besarnya ruang pencarian: jika setiap kendaraan dapat berada di luar kapal atau berada pada salah satu dari W × L posisi dengan dua orientasi, maka tiap kendaraan memiliki paling banyak 2WL + 1 pilihan penempatan, dan ruang pencarian berukuran kasar $(2WL + 1)^n$ sebelum constraint diterapkan. Untuk geladak 10 × 20 dan 25 kendaraan (konfigurasi *default*), batas atas ini ber-orde 10^{65}, jauh di luar jangkauan pencarian menyeluruh.

Pendekatan *local search* cocok karena *state* mudah dibangkitkan secara acak, perpindahan antar state didefinisikan oleh operasi sederhana (*swap, move, rotate*), dan nilai *state* dapat dihitung cepat. *Hill Climbing* dipilih sebagai titik awal karena merupakan algoritma local search paling dasar: perilakunya mudah dianalisis, sehingga menjadi tolok ukur (*baseline*) yang jelas bagi algoritma yang lebih canggih seperti *Simulated Annealing* dan *Genetic Algorithm*.

1.2 Definisi dan Batasan Persoalan

Geladak dimodelkan sebagai *grid* dua dimensi berukuran W × L sel dengan kapasitas berat maksimum C. Setiap kendaraan memiliki id, dimensi (w, l), *ShippingFee*, berat, dan ETA, dan ditempatkan pada koordinat bilangan bulat (x, y)

dengan orientasi *Horizontal* (menempati w × l sel) atau *Vertical* (menempati l × w sel). Solusi menentukan kendaraan mana yang diangkut beserta posisi dan orientasinya, dengan tujuan memaksimalkan total *ShippingFee*. Tidak semua kendaraan wajib diangkut. Sebuah susunan dinyatakan valid jika:

1. tidak ada dua kendaraan yang menempati sel yang sama (tidak tumpang tindih),
2. seluruh sel kendaraan berada di dalam batas geladak, dan
3. total berat kendaraan yang diangkut tidak melebihi kapasitas kapal.

Pencarian dimulai dari susunan acak yang valid. Perpindahan ke susunan tetangga dilakukan dengan tiga operasi: menukar dua kendaraan (*Swap*), memindahkan kendaraan ke koordinat lain atau mengeluarkannya dari kapal (*Move*), dan mengubah orientasi kendaraan (*Rotate*). Atribut ETA disimpan pada data kendaraan tetapi tidak digunakan pada fungsi objektif maupun *constraint*, karena spesifikasi tidak mendefinisikan perannya.

1.3 Tujuan

1. Memodelkan masalah penempatan kendaraan sebagai masalah *local search* (*state*, fungsi objektif, *constraint*, dan fungsi *successor*).
2. Mengimplementasikan empat algoritma: *Steepest Ascent Hill Climbing*, *Hill Climbing with Sideways Move*, *Simulated Annealing*, dan *Genetic Algorithm*.
3. Membandingkan kualitas solusi, konsistensi hasil, dan durasi pencarian, termasuk kapan masing-masing algoritma terjebak di *local optimum* atau *plateau*.

BAB II
IMPLEMENTASI ALGORITMA

2.1 Representasi State

Pada tugas besar ini, state merepresentasikan susunan kendaraan pada geladak kapal. Setiap kendaraan memiliki informasi berupa dimensi, berat, *shipping fee*, dan ETA. Informasi posisi dan orientasi kendaraan disimpan dalam objek `Placement` sedangkan seluruh penempatan kendaraan dalam satu solusi disimpan dalam objek `State`.

Tabel 2.1 Struktur Data Utama

Kelas	Deskripsi
`Vehicle`	Menyimpan identitas, lebar, panjang, ShippingFee, berat, dan ETA setiap kendaraan.
`Ship`	Menyimpan ukuran geladak serta kapasitas berat maksimum kapal.
`Placement`	Menyimpan koordinat (x, y) dan orientasi kendaraan, yaitu horizontal atau vertikal. Kendaraan yang tidak diangkut memiliki koordinat None.
`State`	Menyimpan penempatan seluruh kendaraan, total berat, total ShippingFee, dan informasi sel geladak yang ditempati.
`Problem`	Mengatur data kapal dan kendaraan serta menyediakan fungsi untuk memeriksa validitas penempatan dan menghasilkan tetangga.

Geladak kapal direpresentasikan sebagai *grid* dua dimensi. Setiap kendaraan menempati beberapa sel sesuai dengan dimensi dan orientasi yang dimiliki.

State awal dibangkitkan secara acak melalui fungsi random_state(). Program akan mengacak urutan kendaraan kemudian mencoba menempatkan setiap kendaraan pada posisi dan orientasi acak di dalam batas geladak kapal. Penempatan hanya diterima jika tidak bertumpuk dengan kendaraan lain dan tidak melebihi kapasitas berat kapal. Setiap kendaraan yang tidak berhasil ditempatkan dalam beberapa percobaan tetap ditempatkan di luar kapal.

2.2 Fungsi Objektif dan *Constraint*

Fungsi objektif ini mengukur seberapa baik sebuah state dan dipakai pada semua algoritma untuk membandingkan stte saat ini dengan tetangganya. Nilai yang digunakan adalah total ShippingFee kendaraan yang diangkut:

$$f(s) = \Sigma Z_i F_i$$

Fi merupakan ShippingFee kendaraan ke-i dan Zi merupakan variabel yang memiliki 1 jika kendaran diangkut, 0 jika tidak diangkut. Nili ini disimpan pada atribut `fee` di kelas `state` dan diakses melalui `value`.

Sebuah susunan dikatakan valid jika memenuhi tiga *constraint*:

1. Tidak ada sel geladak yang ditempati lebih dari satu kendaraan.
2. Seluruh kendaraan berada di dalam batas geladak.
3. Total berat kendaraan yang diangkut tidak melebihi kapasitas kapal.

Ketiga constraint diperiksa oleh `is_valid()` yang dipanggil `evaluate()`. Fungsi ini mengembalikan None jika ada pelanggaran dan total `ShippingFee` baru jika valid. Perubahan yang tidak valid langsung ditolak tanpa ada penalti. Nilai baru dihitung dari kendaraan yang berubah tanpa membangun ulang seluruh state.

2.3 Fungsi *Successor* dan Pergerakan Tetangga

Tetangga dari sebuah state dihasilkan oleh `feasible_neighbors()` (untuk *Hill Climbing*) dalam bentuk objek `Move` dan hanya tetangga yang lolos `evaluate()` yang dipakai.

Tabel 2.2 Jenis Perubahan

Jenis Perubahan	Deskripsi
Swap	Menukar posisi dua kendaraan, dengan orientasi masing-masing tetap. Dua kendaraan yang sama-sama di luar kapal tidak ditukar.
Move	Memindahkan kendaraan ke koordinat lain di dalam geladak, memasukkan kendaraan dari luar kapal, atau mengeluarkan kendaraan dari kapal (Move out).
Rotate	Mengubah orientasi horizontal atau vertikal, hanya untuk kendaraan yang lebar dan panjangnya berbeda.

Perubahan posisi atau orientasi kendaraan tidak mengubah total `ShippingFee` selama kendaraan yang diangkut tetap sama. Hal ini membuat beberapa state berbeda bisa memiliki nilai objektif yang sama (plateau). Kondisi ini terjadi pada *Simulated Annealing* dan *Genetic Algorithm* karena keduanya menggunakan mekanisme perubahan solusi melalui `pick_neighbor()` dan `mutate()`.

2.4 Implementasi Varian Hill Climbing

Program mengimplementasikan dua varian *Hill Climbing*, yaitu *Steepest Ascent* dan *Hill Climbing* dengan *Sideways Move*. Kedua algoritma menggunakan fungsi objektif, *constraint*, dan mekanisme pembentukan tetangga yang sama. Perbedaannya terletak pada aturan untuk menerima perpindahan.

Kedua varian dijalankan melalui fungsi internal `_climb()`. Fungsi ini mencatat nilai objektif selama pencarian, jumlah iterasi, jumlah tetangga yang dievaluasi, jumlah langkah *sideways*, waktu eksekusi, dan alasan penghentian iterasi.

2.4.1 Steepest Ascent Hill Climbing

Steepest ascent diimplementasikan pada fungsi steepest_ascent() di hill_climbing.py yang memanggil `_climb()` dengan `max_sideways = 0`. Pada setiap iterasi, seluruh tetangga yang memungkinkan dievaluasi dengan `feasible_neighbors()` dan dipilih yang nilainya paling tinggi (jika nilai seri, dipilih acak). Algoritma berpindah hanya jika nilai tetangga terbaik lebih besar dari nilai *current state*. Jika tetangga terbaik bernilai sama atau tidak ada yang berhenti maka pencarian berhenti dengan alasan `local_optimum`. Pencarian juga berhenti ketika `max_iters` tercapai.

Tabel 2.3 Implementasi *Hill Climbing* dengan *Steepest Ascent*

Kategori	Nama	Deskripsi
Parameter	`problem`	Objek Problem berisi kapal dan daftar kendaraan.
	`initial`	*State* awal (hasil `random_state`).
	`seed`	Seed `random.Random` untuk memilih acak di antara tetangga terbaik yang nilainya seri (*default* 0).
	`max_iters`	Batas iterasi maksimum (*default* 10000).
	`trace`	List opsional untuk menyimpan iterasi dan state untuk *visualizer*.
Variabel	`state`	State saat ini, berubah setiap kali ada langkah naik.
	`cur`	Nilai objektif (`state.value`, yaitu total *fee*) pada awal iterasi.
	`best_val, best_move`	Nilai dan *move* tetangga terbaik pada iterasi ini. Tetangga dengan nilai lebih rendah dari `cur` diabaikan.
	`n_best`	Jumlah tetangga yang seri pada `best_val`, dipakai untuk pemilihan acak.
	`evaluated`	Total tetangga yang dievaluasi.
	`history`	Nilai objektif tiap iterasi, indeks 0 adalah state awal.

	`stop_reason`	Alasan berhenti yaitu, `local_optimum` atau `max_iterations`.
Method	`steepest_ascent(...)`	Pembungkus yang memanggil `_climb` dengan `max_sideways = 0`.
	`_climb(...)`	Loop utama hill climbing yang mengembalikan `HillClimbingResult`.
	`problem.feasible_neighbors(state)`	Generator semua tetangga yang memungkinkan beserta nilainya (Swap, Move (termasuk Move out), dan Rotate).
	`problem.evaluate(state, change)`	Menghitung *fee* state baru secara delta; None jika overlap, keluar geladak, atau melebihi kapasitas.
	`state.apply(move)`	Membuat State baru dengan menerapkan perubahan placement dari move.

Algoritma

```
function SteepestAscent(problem, initial, max_iters):
    state <- initial
    for i <- 1 to max_iters:
        cur <- state.value
        best_val <- None;  best_move <- None;  n_best <- 0
        for each (move, val) in feasible_neighbors(state):
            if val < cur: continue
            if best_val = None or val > best_val:
                best_val <- val;  best_move <- move;  n_best <- 1
            else if val = best_val:
                n_best <- n_best + 1
                with probability 1/n_best: best_move <- move
        if best_val = None or best_val = cur:    // tidak ada tetangga
yang lebih baik
            stop (local_optimum)
        state <- apply(state, best_move)         // best_val > cur
    return state
```

2.4.2 *Hill Climbing* dengan *Sideways Move*

Sideways move merupakan variasi dari *Hill Climbing* yang memperbolehkan pindah ke tetangga dengan nilai yang sama dengan *current state*. Pada kode ini, keduanya menggunakan fungsi yang sama, `_climb`: `Steepest Ascent` adalah kasus `max_sideways = 0`, sedangkan `sideways_move` menggunakan `max_sideways = 100` secara *default*. Pada tiap iterasi, algoritma mengevaluasi semua tetangga yang memungkinkan, memilih yang nilainya paling tinggi (jika seri, dipilih secara acak), lalu bergerak naik jika ada tetangga dengan nilai yang lebih baik. Jika tetangga terbaik bernilai sama, algoritma tetap berpindah meskipun belum terjadi peningkatan nilai. Perpindahan pada nilai yang sama ini dapat tetap terjadi selama `sideways_streak` belum mencapai `max_sideways`. Counter di-reset ke 0 setiap kali terdapat langkah naik.

Pencarian berhenti jika tidak terdapat tetangga yang sama atau lebih baik, jika batas *sideways* tercapai, atau sudah mencapai `max_iters`.

Tabel 2.4 Implementasi Hill Climbing dengan Sideways Move

Kategori	Nama	Deskripsi
Parameter	`problem`	Objek Problem berisi kapal (Ship) dan daftar kendaraan.
	`initial`	State awal (hasil `random_state`).
	`seed`	Seed `random.Random` untuk memilih acak di antara tetangga yang nilainya sama.
	`max_sideways`	Batas langkah *sideways* berturut-turut (*default* 100). Jika bernilai 0 berarti *Steepest Ascent*.
	`max_iters`	Batas iterasi maksimum (*default* 10000).
	`trace`	List opsional untuk menyimpan iterasi dan state untuk visualizer.
Variabel	`state`	State saat ini, berubah tiap langkah naik atau *sideways*.
	`cur`	Nilai objektif (`state.value`, yaitu total *fee*) pada awal iterasi.
	`best_val, best_move`	Nilai dan *move* tetangga terbaik pada iterasi ini. Tetangga dengan nilai lebih rendah dari `cur` diabaikan.
	`n_best`	Jumlah tetangga yang seri pada `best_val`, dipakai untuk pemilihan acak.
	`sideways_streak`	Jumlah langkah sideways berturut-turut. Di-*reset* ke 0 saat ada langkah naik.
	`sideways_total`	Total seluruh langkah *sideways* selama pencarian.
	`evaluated`	Total tetangga yang dievaluasi.
	`history`	Nilai objektif tiap iterasi, indeks 0 adalah state awal.
	`stop_reason`	Alasan berhenti yaitu, `local_optimum`, `max_sideways_reached`, atau `max_iterations`.

Method	`sideways_move(...)`	Pembungkus yang memanggil `_climb` dengan `max_sideways` yang diberikan.
	`_climb(...)`	Loop utama hill climbing yang mengembalikan `HillClimbingResult`.
	`problem.feasible_neighbors(state)`	Generator semua tetangga yang memungkinkan beserta nilainya (Swap, Move (termasuk Move out), dan Rotate).
	`problem.evaluate(state, change)`	Menghitung *fee* state baru secara delta; mengembalikan None jika *overlap*, keluar geladak, atau melebihi kapasitas.
	`state.apply(move)`	Membuat State baru dengan menerapkan perubahan placement dari move.

Algoritma

```
function SidewaysMove(problem, initial, max_sideways, max_iters):
    state <- initial
    sideways_streak <- 0
    for i <- 1 to max_iters:
        cur <- state.value
        best_val <- None;  best_move <- None;  n_best <- 0
        for each (move, val) in feasible_neighbors(state):
            if val < cur: continue
            if best_val = None or val > best_val:
                best_val <- val;  best_move <- move;  n_best <- 1
            else if val = best_val:
                n_best <- n_best + 1
                with probability 1/n_best: best_move <- move
        if best_val = None:                          // tidak ada tetangga >=
cur
            stop (local_optimum)
        else if best_val > cur:                      // langkah naik
            state <- apply(state, best_move)
            sideways_streak <- 0
        else:                                        // best_val = cur,
langkah sideways
            if sideways_streak >= max_sideways: stop
(max_sideways_reached)
            state <- apply(state, best_move)
            sideways_streak <- sideways_streak + 1
    return state
```

2.5 Implementasi *Simulated Annealing*

Simulated annealing (SA) pada kode ini membangkitkan satu tetangga secara acak per iterasi melalui pick_neighbor (rotate, move, atau swap dengan peluang yang sama sampai 100 percobaan sampai menemukan tetangga yang memungkikan). Jika tetangga memiliki nilai yang lebih baik ($\Delta E > 0$) maka langsung diterima. Jika lebih buruk ($\Delta E < 0$) maka akan diterima dengan peluang:

$$P = e^{\frac{\Delta E}{T}}$$

Dimana nilai ΔE merupakan pengurangan antara state saat ini (*current state*) dengan state tetangga yang dipilih secara acak (*neighbor state*). Suhu diturunkan secara geometrik, `T <- T* cooling rate` dan pencarian berhenti ketika `T<= stopping_temp`. Dengan nilai default (`init_temp = 50, cooling_rate = 0.999, stopping_temp = 0.01`) dan jumlah iterasi tetap. Current state bisa memburuk sehingga state terbaik disimpan terpisah di variabel best dan dikembalikan sebagai `final_state`. Selain itu, kode mencatat jumlah langkah buruk yang diterima dan mendeteksi kondisi stuck, jika current tidak berubah selama `stuck_window` iterasi maka akan dicek sekali dengan `is_local_optimum` apakah memang tidak ada tetangga yang lebih baik.

Tabel 2.5 Implementasi Simulated Annealing

Kategori	Nama	Deskripsi
Parameter	`problem`	Objek Problem berisi kapal dan daftar kendaraan.
	`initial`	State awal (hasil `random_state`).
	`seed`	Seed `random.Random` untuk pemilihan tetangga dan keputusan menerima langkah buruk.
	`init_temp`	Suhu awal (*default* 50.0).
	`cooling_rate`	Faktor pengali penurunan suhu per iterasi, harus di antara 0 dan 1 (*default* 0.999).
	`stopping_temp`	Suhu berhenti, harus > 0 dan < `init_temp` (default 0.01).
	`stuck_window`	Jumlah iterasi tanpa perubahan nilai sebelum dicek apakah sudah di *local optimum* (*default* 50).
	`trace`	List opsional untuk menyimpan (iterasi, state) setiap kali `best` membaik.
Variabel	`current`	State saat ini, bisa lebih buruk dari `best` karena langkah buruk diterima.
	`best`	State dengan nilai tertinggi yang pernah ditemukan dan dikembalikan sebagai `final_state`.

	`T`	Suhu saat ini, dikalikan dengan `cooling_rate` tiap iterasi.
	`delta`	Selisih nilai tetangga dan `current` (ΔE).
	`probability`	1.0 jika delta > 0, selain itu exp(delta / T).
	`iteration`	Penghitung iterasi.
	`accepted, worse_accepted`	Jumlah langkah yang diterima dan yang di antaranya lebih buruk.
	`neighbors_evaluated`	Jumlah iterasi yang berhasil mendapat tetangga *feasible*.
	`consec_no_improve, stuck_checked`	Penghitung iterasi tanpa perubahan nilai dan penanda agar cek *local optimum* hanya sekali per episode stagnan.
	`stuck_count, stuck_iters`	Berapa kali dan pada iterasi ke berapa SA terdeteksi *stuck* di *local optimum*.
	`current_hist, best_hist, temp_hist`	Riwayat nilai `current`, nilai `best`, dan suhu per iterasi (untuk grafik).
	`exp_hist`	Log (iterasi, delta, probabilitas) tiap langkah yang lebih buruk, untuk plot nilai e^(ΔE/T).
Method	`simulated_annealing(...)`	Fungsi utama untuk memvalidasi parameter lalu menjalankan loop pendinginan. Fungsi ini engembalikan `AnnealingResult`.
	`pick_neighbor(problem, state, rng)`	Membangkitkan satu tetangga acak (Rotate, Move, atau Swap) dan nilainya, None jika 100 percobaan tidak ada yang *feasible*.
	`is_local_optimum(problem, state)`	True jika tidak ada tetangga *feasible* dengan nilai lebih besar dari `state.value`.
	`problem.evaluate(state, change)`	Menghitung *fee* state baru secara delta, None jika melanggar batasan.

	`state.apply(move)`	Membuat State baru setelah *move* diterapkan.

Algoritma

```
function SimulatedAnnealing(problem, initial, init_temp, cooling_rate,
stopping_temp):
    current <- initial;  best <- initial
    T <- init_temp
    while T > stopping_temp:
        neighbor <- pick_neighbor(current)   // Rotate / Move / Swap acak
if neighbor != None:
            delta <- neighbor.value - current.value
            if delta > 0:  p <- 1
            else:          p <- exp(delta / T)
            if delta > 0 or random() < p:
                current <- neighbor
                if current.value > best.value: best <- current
        T <- T * cooling_rate
    return best
```

2.6 Implementasi Genetic Algorithm

Genetic algorithm (GA) pada kose ini mengelola populasi berisi `population_size` state (*default* 50) yang dibangkitkan secara acak dengan `random_state()`. Pada setiap generasi, `elite_size` state terbaik langsung dipertahankan dan sisa populasi diisi anak. Setiap anak dibentuk dengan memilih dua induk melalui `random_selection`, lalu menggabungkannya dengan *crossover* yang memiliki peluang `crossover_rate` (tanpa ini, anak merupakan salinan dari induk pertama). Kemudian, anak dimutasi sampai `mutation_steps` kali oleh `mutate` dengan peluang `mutation_rate`. Satu generasi dihitung sebagai satu iterasi dan pencarian berhenti setelah `generations` generasi (*default* 300). Individu terbaik sepanjang pencarian disimpan terpisah di terbaik dan dikembalikan sebagai `final_state`. Selain itu, pada algoritma yang dibangun, kode mencatat nilai terbaik dan rata-rata nilai populasi pada setiap generasi/iterasi (`history_best` dan `history_avg`).

Tabel 2.6 Implementasi Genetic Algorithm

Kategori	Nama	Deskripsi
Parameter	`problem`	Objek Problem berisi kapal dan daftar kendaraan.
	`population_size`	Jumlah individu per generasi (*default* 50).
	`generations`	Jumlah generasi, yaitu banyak iterasi (*default* 300).

	`seed`	Seed `random.Random` agar hasil dapat diulang.
	`crossover_rate`	Peluang dua induk di-*crossover* (*default* 0,9).
	`mutation_rate`	Peluang anak dimutasi (*default* 0,9).
	`mutation_steps`	Maksimum jumlah mutasi per anak (*default* 6).
	`elite_size`	Jumlah individu terbaik yang langsung lolos ke generasi berikutnya (*default* 2).
	`tournament_k`	Ukuran turnamen pada seleksi induk (*default* 2).
	`init_attempts`	Jumlah percobaan menempatkan tiap kendaraan saat membangkitkan state awal (*default* 40).
	`trace`	List opsional untuk menyimpan (generasi, state terbaik) setiap kali terbaik membaik.
Variabel	`populasi`	Kumpulan state pada generasi saat ini.
	`terbaik`	State bernilai tertinggi sepanjang pencarian, dikembalikan sebagai `final_state`.
	`awal`	Individu terbaik pada populasi awal menjadi `initial_state`.
	`urut, baru`	Populasi yang diurutkan menurut nilai dan populasi generasi berikutnya yang dimulai dari individu elit.
	`p1, p2, anak`	Dua induk hasil seleksi dan anak yang dihasilkan.
	`history_best, history_avg`	Nilai terbaik sepanjang pencarian dan rata-rata nilai populasi pada tiap generasi (indeks 0 adalah populasi awal).
	`MUTATION_WEIGHTS`	Konstanta bobot jenis mutasi (Swap 0,27, Move 0,53, Rotate 0,20).
	`sumbu, garis`	Pada *crossover*, sumbu pemotongan (lebar atau panjang geladak) dan posisi garis potong.
	`kiri, kanan`	Pada *crossover*, induk yang menyumbang sisi awal dan sisi akhir garis potong.

	`candidates`	Pada *crossover*, kendaraan yang diwariskan dari kedua induk diurutkan dari *fee* terbesar.
	`occupied_cells, total_weight`	Pada *crossover*, sel geladak dan total berat yang sudah terisi oleh anak.
Method	`genetic_algorithm(...)`	Fungsi utama yang menjalankan seluruh generasi dan mengembalikan `GeneticResult`.
	`random_selection(population, rng, k)`	Tournament selection, mengambil k individu acak dan mengembalikan yang bernilai tertinggi.
	`crossover(problem, a, b, rng)`	Menggabungkan dua induk lewat garis potong menjadi satu anak yang valid.
	`add_vehicle(...)`	Fungsi di dalam *crossover* yang memasukkan kendaraan ke anak jika tidak *overlap* di dalam geladak dan tidak melebihi kapasitas.
	`mutate(problem, state, rng, tries)`	Menerapkan satu perubahan valid (Swap, Move, atau Rotate) pada state.
	`problem.random_state(rng, attempts)`	Membangkitkan state acak yang valid untuk populasi awal.
	`problem.evaluate(state, change)`	Memeriksa validitas perubahan pada mutasi, None jika melanggar batasan.
	`state.apply(move)`	Membuat State baru setelah perubahan diterapkan.

Algoritma

```
function GeneticAlgorithm(problem, population_size, generations,
crossover_rate,
                          mutation_rate, mutation_steps, elite_size,
tournament_k):
    populasi <- population_size state acak yang valid
    terbaik  <- individu bernilai tertinggi di populasi
    for gen <- 1 to generations:
        urut <- populasi diurutkan dari nilai tertinggi
        baru <- elite_size individu pertama dari urut
        while |baru| < population_size:
            p1 <- TournamentSelection(populasi, tournament_k)
            p2 <- TournamentSelection(populasi, tournament_k)
            if random() < crossover_rate: anak <- Crossover(p1, p2)
            else:                         anak <- p1
            if random() < mutation_rate:
                repeat (acak 1 .. mutation_steps) kali: anak <-
Mutate(anak)
            tambahkan anak ke baru
        populasi <- baru
```

```
        terbaik  <- yang lebih tinggi antara terbaik dan individu terbaik
di populasi
    return terbaik

function Crossover(a, b):
    sumbu <- acak (lebar atau panjang geladak);  garis <- acak di antara
1 dan (panjang sumbu - 1)
    kiri, kanan <- (a, b) atau (b, a) secara acak
    kandidat <- kendaraan kiri yang seluruhnya di sisi awal garis
              + kendaraan kanan yang seluruhnya di sisi akhir garis
    for tiap kandidat, dari fee terbesar:
        masukkan ke anak jika tidak overlap, di dalam geladak, dan berat
total <= kapasitas
    for tiap kendaraan yang belum masuk, dari fee terbesar:
        coba penempatannya pada a, lalu pada b, dengan pemeriksaan yang
sama
    kendaraan yang tetap belum masuk berada di luar kapal
    return anak
```

BAB III
HASIL PENGUJIAN DAN ANALISIS

3.1 Konfigurasi Eksperimen

Semua algoritma dijalankan pada satu *instance* masalah yang sama, dibuat oleh `generate_problem`. Tiap konfigurasi dijalankan 3 kali dengan seed 0, 1, dan 2. Untuk *Steepest Ascent, Sideways Move*, dan *Simulated Annealing*, state awal percobaan ke-k identik di ketiga algoritma (hasil random_state dengan seed yang sama). *Genetic Algorithm* membangun populasi awalnya sendiri.

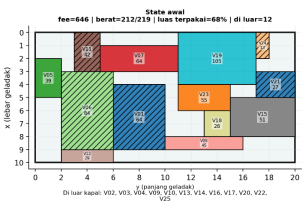
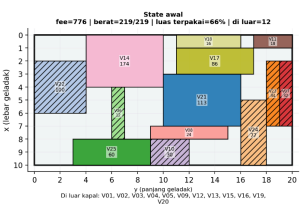
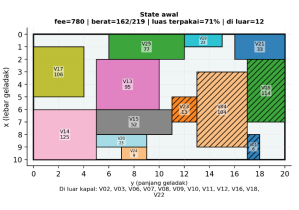
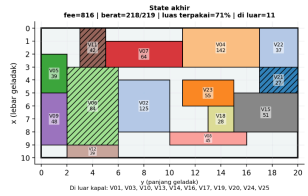
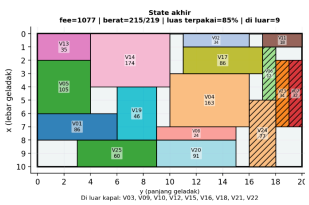
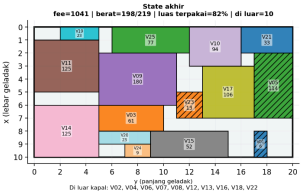
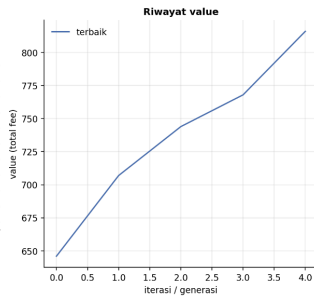
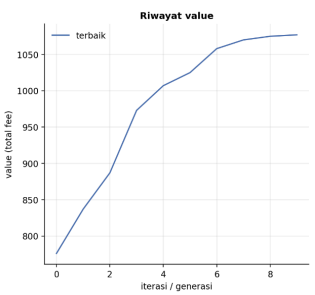
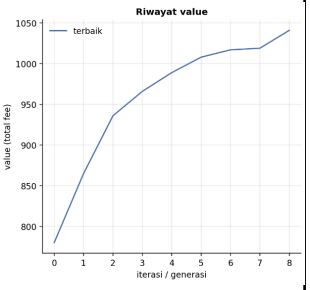
Tabel 3. 1 Konfigurasi eksperimen

Komponen	Nilai
Jumlah kendaraan	25
Geladak kapal	10 × 20
Kapasitas berat kapal	219
Total fee seluruh kendaraan	1705
Batas atas objective (relaksasi)	1355
Pembangkit instance	generate_problem(n=25, 10×20, seed=1)
Jumlah percobaan	3 per konfigurasi (seed 0, 1, 2)
Steepest Ascent	max_iters = 10000
Sideways Move	max_sideways = 100, max_iters = 10000
Simulated Annealing	T awal 50, cooling rate 0.999, T berhenti 0.01, stuck window 50 iterasi
Genetic Algorithm (parameter tetap)	crossover 0.9, mutasi 0.9, maks 6 mutasi per anak, elitisme 2, turnamen k = 2
Genetic Algorithm (variasi iterasi)	populasi dikontrol = 50 (generasi 20, 100, 300)
Genetic Algorithm (variasi populasi)	generasi dikontrol = 100 (populasi 10, 70, 150)

3.2 Hasil Eksperimen

3.2.1 Steepest Ascent Hill Climbing

Tabel 3.2 Hasil Eksperimen Steepest Ascent Hill Climbing

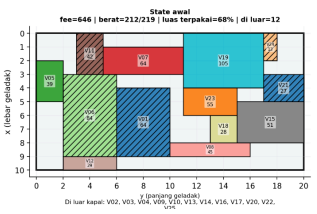
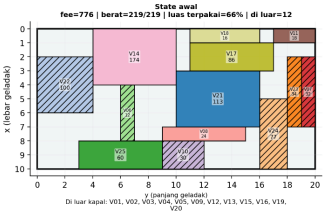
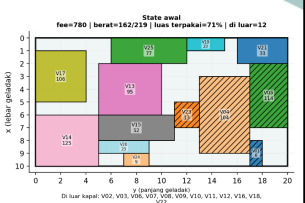
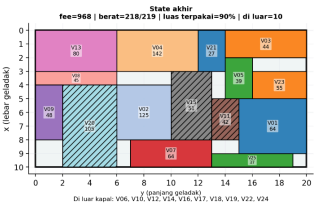
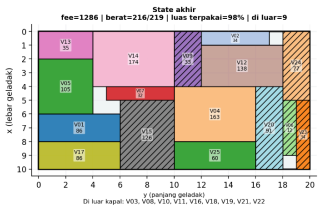
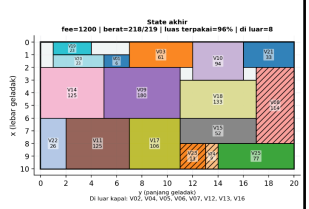
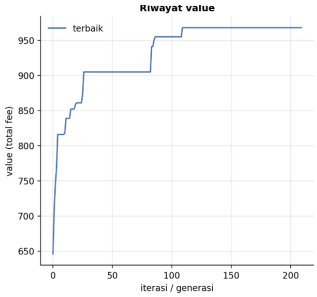
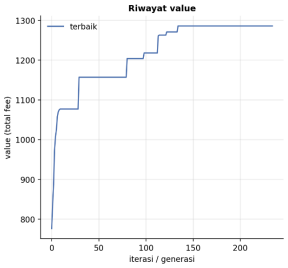
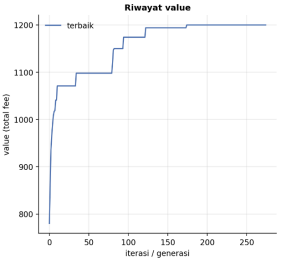
Komponen	Percobaan 1	Percobaan 2	Percobaan 3
State awal			
State akhir			
Nilai objective (awal → akhir)	646 → 816	776 → 1077	780 → 1041
Objective vs iterasi			
Banyak iterasi	4	9	8
Alasan berhenti	local_optimum	local_optimum	local_optimum
Durasi pencarian	0.1083 detik	0.0744 detik	0.1515 detik

3.2.2 Hill Climbing dengan Sideways Move

Pada percobaan dengan algoritma ini digunakan maximum sideways move = 100.

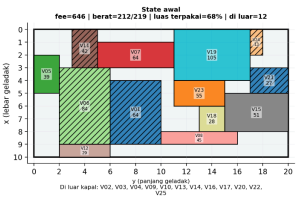
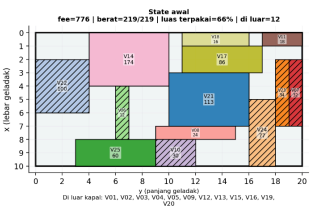
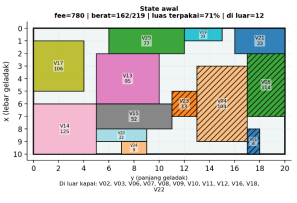
Tabel 3.3 Hasil Eksperimen Hill Climbing dengan Sideways Move

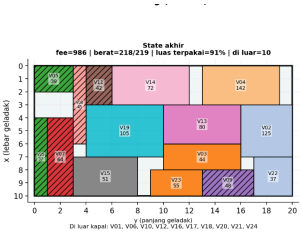
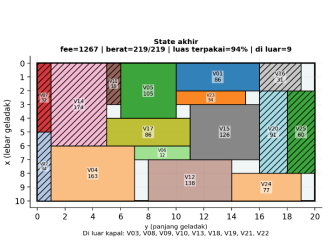
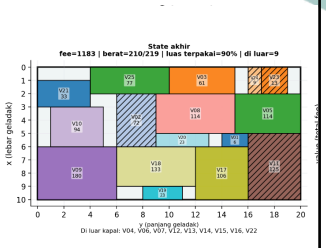
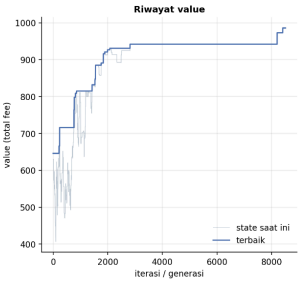
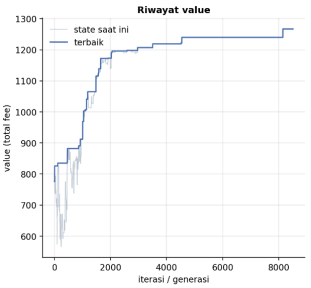
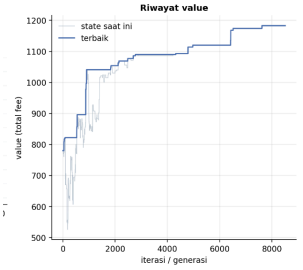
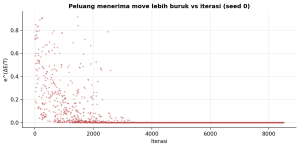
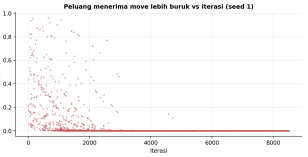
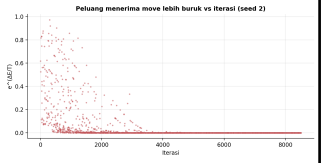
Komponen	Percobaan 1	Percobaan 2	Percobaan 3

State awal			
State akhir			
Nilai objective (awal → akhir)	646 → 968	776 → 1286	780 → 1200
Objective vs iterasi			
Banyak iterasi	209	234	274
Alasan berhenti	max_sideways_reached	max_sideways_reached	max_sideways_reached
Durasi pencarian	2.0994 detik	2.0434 detik	5.1248 detik

3.2.3 Simulated Annealing

Tabel 3.4 Hasil Eksperimen Simulated Annealing

Komponen	Percobaan 1	Percobaan 2	Percobaan 3
State awal			

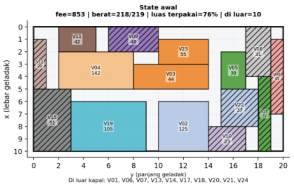
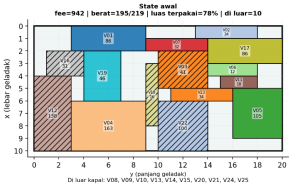
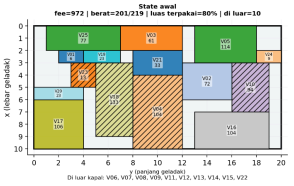
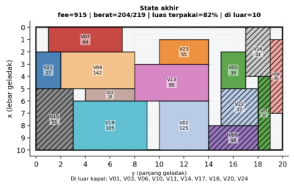
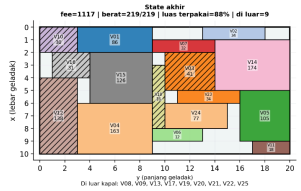
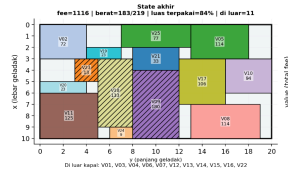
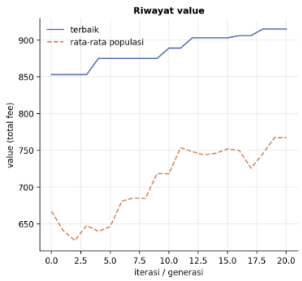
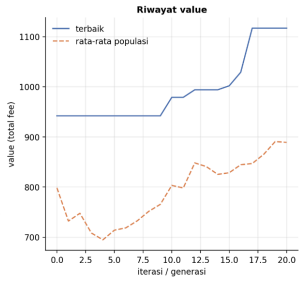
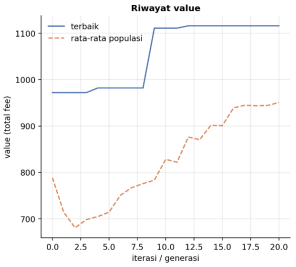
State akhir (terbaik)			
Nilai objective (awal → akhir)	646 → 986	776 → 1267	780 → 1183
Objective vs iterasi			
Plot e^(ΔE/T) vs iterasi			
Frekuensi stuck di local optimum	4	2	6
Banyak iterasi	8513	8513	8513
Durasi pencarian	0.5756 detik	0.8137 detik	0.7796 detik

3.2.4 Genetic Algorithm

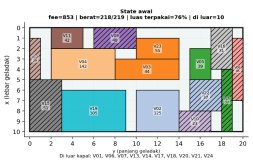
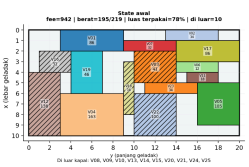
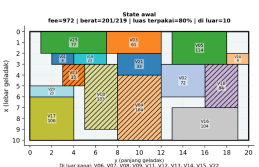
Pada percobaan ini dilakukan dengan beberapa skema. **Skema A** dengan populasi dikontrol (50), variasi banyak iterasi (generasi) 20, 100, dan 300. **Skema B** dengan iterasi dikontrol (100), variasi populasi 10, 70, dan 150. State awal pada tabel adalah individu terbaik dari populasi awal, bukan state acak tunggal.

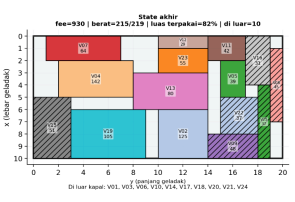
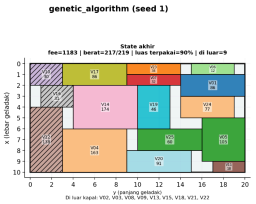
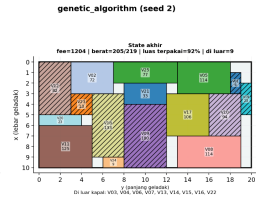
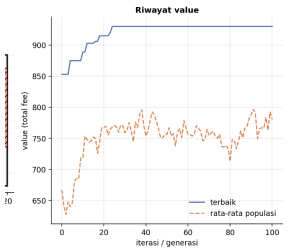
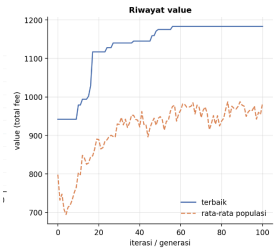
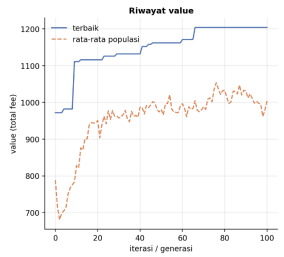
Tabel 3.5 Hasil Eksperimen Genetic Algorithm (Populasi 50 dan iterasi 20)

Komponen	Percobaan 1	Percobaan 2	Percobaan 3

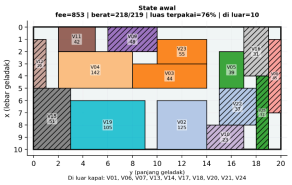
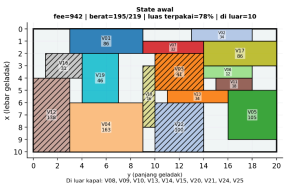
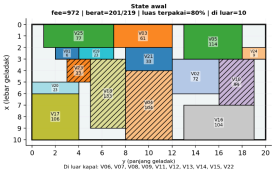
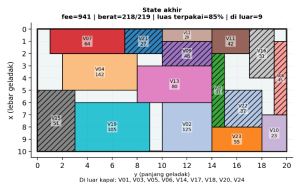
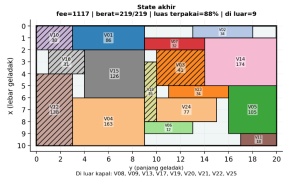
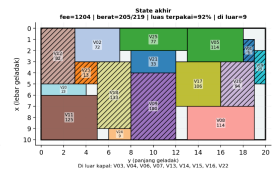
State awal			
State akhir			
Nilai objective (awal → akhir)	853 → 915	942 → 1117	972 → 1116
Objective maks dan rata-rata vs iterasi			
Jumlah populasi	50	50	50
Banyak iterasi	20	20	20
Durasi pencarian	1.34 detik	0.64s	0.58s

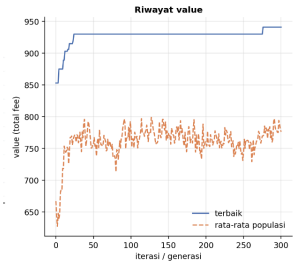
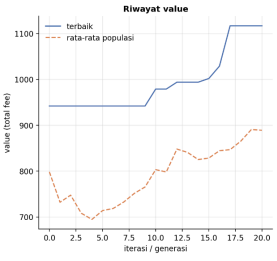
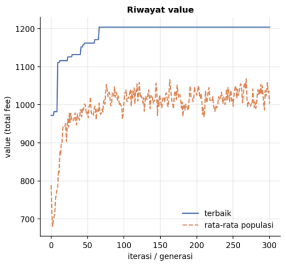
Tabel 3.6 Hasil Eksperimen Genetic Algorithm (Populasi 50 dan iterasi 100)

Komponen	Percobaan 1	Percobaan 2	Percobaan 3
State awal			

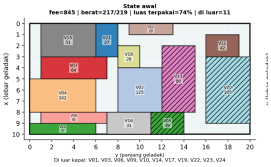
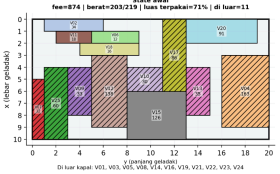
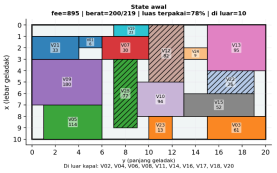
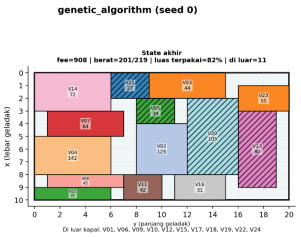
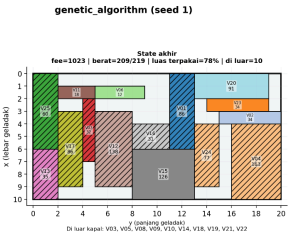
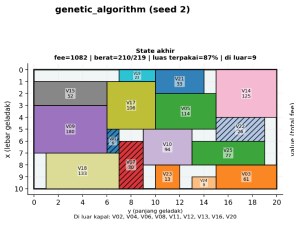
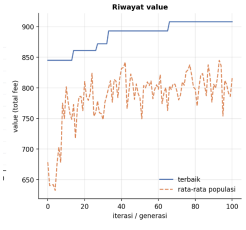
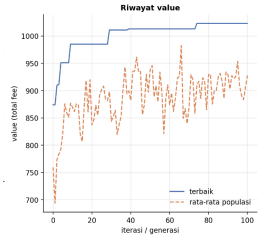
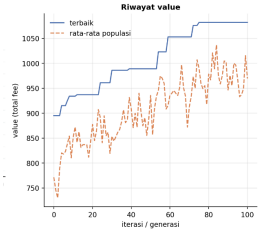
State akhir			
Nilai objective (awal → akhir)	853 → 930	942 → 1183	972 → 1204
Objective maks dan rata-rata vs iterasi			
Jumlah populasi	50	50	50
Banyak iterasi	100	100	100
Durasi pencarian	0.70s	0.64s	0.65s

Tabel 3.7 Hasil Eksperimen Genetic Algorithm (Populasi 50 dan iterasi 300)

Komponen	Percobaan 1	Percobaan 2	Percobaan 3
State awal			
State akhir			
Nilai objective (awal → akhir)	853 → 941	942 → 1117	972 → 1204

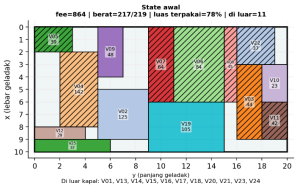
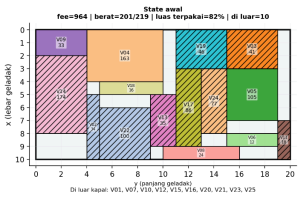
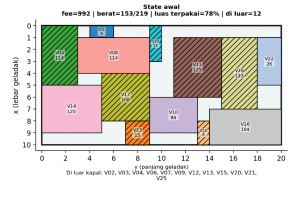
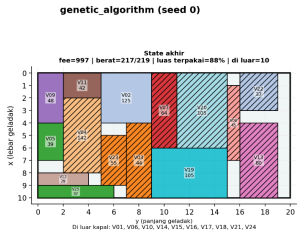
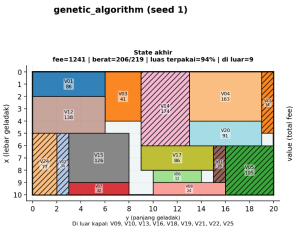
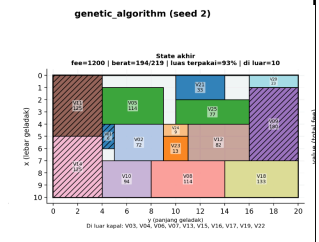
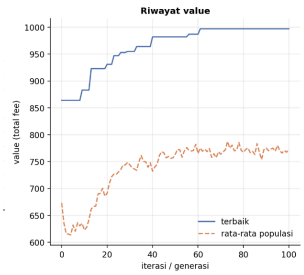
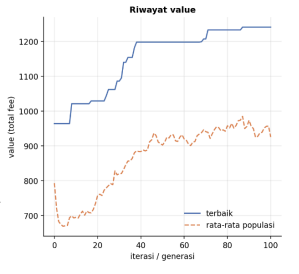
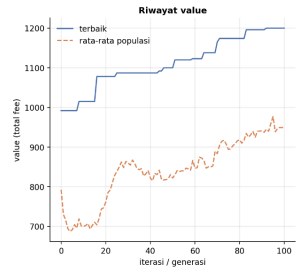
Objective maks dan rata-rata vs iterasi			
Jumlah populasi	50	50	50
Banyak iterasi	100	100	100
Durasi pencarian	0.66s	0.62 detik	0.60 detik

Tabel 3.8 Hasil Eksperimen Genetic Algorithm (Populasi 10 dan Iterasi 100)

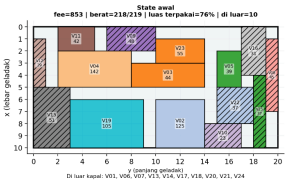
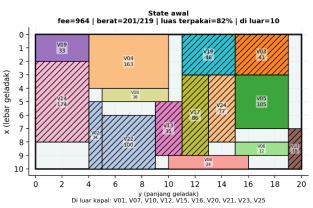
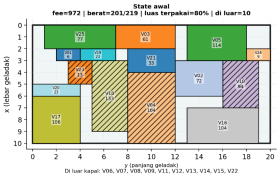
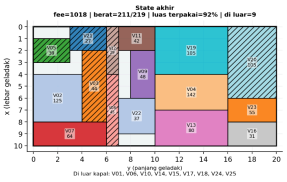
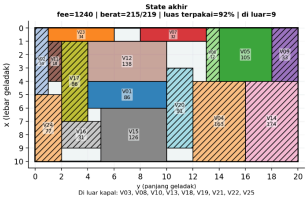
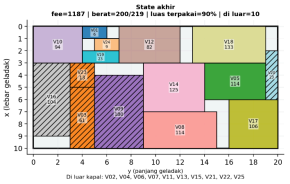
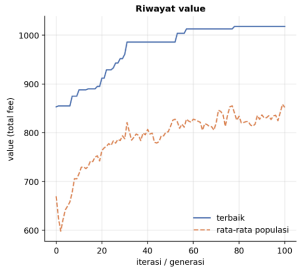
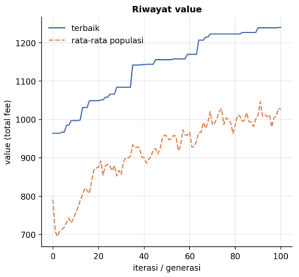
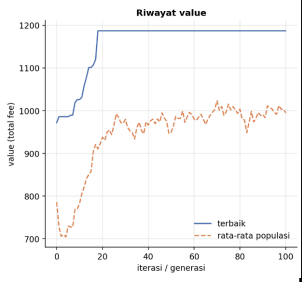
Komponen	Percobaan 1	Percobaan 2	Percobaan 3
State awal			
State akhir			
Nilai objective (awal → akhir)	845 → 908	874 → 1023	895 → 1082c
Objective maks dan rata-rata vs iterasi			

Jumlah populasi	10	10	10
Banyak iterasi	100	100	100
Durasi pencarian	0.4043 detik	0.4809 detik	0.7542 detik

Tabel 3.9 Hasil Eksperimen Genetic Algorithm (Populasi 150 dan Iterasi 100)

Komponen	Percobaan 1	Percobaan 2	Percobaan 3
State awal			
State akhir			
Nilai objective (awal → akhir)	864 → 997	964 → 1241	992 → 1200
Objective maks dan rata-rata vs iterasi			
Jumlah populasi	150	150	150
Banyak iterasi	100	100	100
Durasi pencarian	10.3489 detik	28.1785 detik	12.1189 detik

Tabel 3.10 Hasil Eksperimen Genetic Algorithm (Populasi 70 dan Iterasi 100)

Komponen	Percobaan 1	Percobaan 2	Percobaan 3
State awal			
State akhir			
Nilai objective (awal → akhir)	853 → 1018	964 → 1240	972 → 1187
Objective maks dan rata-rata vs iterasi			
Jumlah populasi	70	70	70
Banyak iterasi	100	100	100
Durasi pencarian	4.8495 detik	6.1686 detik	13.5098 detik

3.3 Analisis Hasil Eksperimen

Tabel 3.11 Analisis Hasil Pertanyaan 1

Pertanyaan 1. Seberapa dekat tiap-tiap algoritma bisa mendekati global optima dan mengapa hasilnya demikian?	
Steepest Ascent Hill-Climbing	Rata-rata 978,0 (72,2% dari batas atas

	1355), terbaik 1077. Berhenti di *local optimum* setelah 4-9 iterasi. Memindahkan atau memutar kendaraan tidak mengubah total *fee* sehingga langkah naik hanya muncul saat ada kendaraan baru yang muat. Tanpa mekanisme melewati *plateau*, algoritma ini cepat terjebak.
Sideways Move Hill-Climbing	Rata-rata 1151,3 (85,0%), terbaik 1286 (94,9%). Mampu menyeberangi area datar (209-274 iterasi). Ketiga percobaan berhenti karena batas 100 langkah *sideways* tercapai, bukan karena *local optimum* jadi pencarian masih berpotensi naik.
Simulated Annealing	Rata-rata 1145,3 (84,5%), terbaik 1267. Langkah buruk diterima dengan peluang e^(ΔE/T) sehingga bisa keluar dari *local optimum*. Terdeteksi *stuck* 2-6 kali, tetapi tetap lanjut sampai 8513 iterasi.
Genetic Algorithm	Bergantung pada populasi, yaitu 1004,3 (populasi 10) sampai 1148,3 (populasi 70, 84,7%). *Crossover* dan mutasi menggabungkan banyak solusi, tetapi populasi kecil cepat kehilangan keragaman (lihat Pertanyaan 5).

Tabel 3.12 Analisis Hasil Pertanyaan 2

Pertanyaan 2. Bagaimana perbandingan hasil pencarian tiap-tiap algoritma dengan algoritma local search yang lain?	
Steepest Ascent Hill-Climbing	Paling rendah (978,0), sekitar 15% di bawah kelompok teratas.
Sideways Move Hill-Climbing	Tertinggi (1151,3). Unggul 173,3 poin dari *Steepest Ascent* pada rata-rata dan menang di ketiga seed (152, 209, dan 159 poin).
Simulated Annealing	1145,3, selisih hanya 6 poin dari *Sideways Move*.
Genetic Algorithm	Populasi 70: 1148,3 dan populasi 150: 1146,0. Keduanya setara dengan *Sideways Move* dan SA.

Tabel 3.13 Analisis Hasil Pertanyaan 3

Pertanyaan 3. Bagaimana perbandingan durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya?	
Steepest Ascent Hill-Climbing	Tercepat, 0,11 detik, karena berhenti setelah hanya 4-9 iterasi.
Sideways Move Hill-Climbing	0,72 detik, tiap iterasi hanya mengevaluasi satu tetangga acak.
Simulated Annealing	3,09 detik. Tiap iterasi mengevaluasi seluruh tetangga dan berjalan 209-274 iterasi, sehingga lebih lambat dari SA.
Genetic Algorithm	Durasi naik seiring populasi × generasi: 0,55 detik (populasi 10), 8,18 detik (populasi 70), 16,88 detik (populasi 150). Percobaan 2 pada populasi 150 (28,18 detik) sekitar 2,3 kali lebih lama dari percobaan lain. Kemungkinan penyebabnya beban sistem, bukan algoritmanya.

Tabel 3.14 Analisis Hasil Pertanyaan 4

Pertanyaan 4. Seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan?	
Steepest Ascent Hill-Climbing	Rentang antar-*seed* 261 poin (816-1077).
Sideways Move Hill-Climbing	Rentang terlebar, 318 poin (968-1286).
Simulated Annealing	Rentang 281 poin (986-1267).
Genetic Algorithm	Rentang 222 (populasi 70), 244 (populasi 150), dan 174 (populasi 10). Populasi 10 paling sempit, tetapi rata-ratanya paling rendah (1004,3), jadi algoritma ini konsisten pada hasil yang kurang baik. Populasi 150 pada seed 0 hanya 997, sedangkan *seed* 1 dan 2 mencapai 1241 dan 1200.

Tabel 3.15 Analisis Hasil Pertanyaan 5

Pertanyaan 5. Bagaimana pengaruh banyak iterasi dan jumlah populasi terhadap hasil akhir pencarian pada *Genetic Algorithm*?	
Banyak iterasi (populasi 50)	Naik dari 20 ke 100 iterasi menaikkan rata-rata dari 1049,3 menjadi 1105,7

	(+56,4). Kurva nilai terbaik mendatar setelah sekitar iterasi ke-30 sampai 60, jadi iterasi tambahan terutama menambah waktu.
Jumlah populasi (iterasi 100)	Rata-rata naik dari 1004,3 (populasi 10) ke 1105,7 (populasi 50) ke 1148,3 (populasi 70), lalu stagnan di 1146,0 (populasi 150) sementara durasi naik dari 8,18 ke 16,88 detik. Populasi kecil cepat kehilangan keragaman, sedangkan populasi besar tidak lagi memberi kombinasi baru yang berguna.
Konfigurasi terbaik	Populasi 70 dengan 100 iterasi memberi keseimbangan terbaik antara kualitas dan waktu.

Tabel 3.16 Analisis Hasil Pertanyaan 6

Pertanyaan 6. Algoritma mana yang paling seimbang antara kualitas hasil dan durasi pencarian?	
Jawab: *Simulated Annealing* merupakan algoritma yang paling seimbang antara kualitas hasil dan durasi pencarian karena menghasilkan kualitas setara dengan waktu tercepat, yaitu 0,72 detik.	
Steepest Ascent Hill-Climbing	6,5 kali lebih cepat dari SA, tetapi hasilnya sekitar 15% lebih rendah.
Sideways Move Hill-Climbing	Kualitas setara SA, tetapi 4,3 kali lebih lambat (3,09 vs 0,72 detik).
Genetic Algorithm	Populasi 70 setara kualitasnya, tetapi 11,3 kali lebih lambat dari SA (8,18 detik).

BAB IV
KESIMPULAN DAN SARAN

4.1 Kesimpulan

Berdasarkan hasil pengujian, belum ada algoritma yang terbukti mencapai solusi optimum global. Nilai terbaik yang diperoleh adalah 1286 dari Hill Climbing dengan Sideways Move, sedangkan batas atas solusi adalah 1355. Steepest Ascent Hill Climbing memiliki waktu pencarian tercepat, tetapi menghasilkan kualitas solusi paling rendah karena mudah terjebak di local optimum.

Sebaliknya, Sideways Move, Simulated Annealing, dan Genetic Algorithm menghasilkan kualitas solusi yang relatif setara dan lebih baik. Di antara ketiganya, Simulated Annealing menjadi pilihan paling seimbang karena menghasilkan kualitas yang setara dengan waktu pencarian paling singkat, yaitu 0,72 detik. Pada Genetic Algorithm, penambahan populasi dari 10 menjadi 70 meningkatkan kualitas solusi, tetapi populasi 150 tidak memberikan peningkatan berarti. Secara keseluruhan, hasil pengujian juga menunjukkan bahwa nilai solusi masih bervariasi antarpercobaan, sehingga hasil terbaik belum tentu selalu diperoleh pada setiap eksekusi.

4.2 Saran

Berdasarkan hasil pengujian, terdapat beberapa hal yang dapat dilakukan untuk meningkatkan kualitas solusi dan membuat perbandingan antaralgoritma lebih akurat.

1. **Menambah jumlah percobaan** menjadi 20–30 kali untuk setiap algoritma dan menggunakan uji statistik agar perbedaan hasil dapat dibandingkan dengan lebih meyakinkan.

2. **Mencari nilai optimum sebenarnya** menggunakan metode Integer Programming atau Branch and Bound agar kualitas solusi yang dihasilkan setiap algoritma dapat dinilai dengan lebih akurat.

3. **Menerapkan random restart pada Hill Climbing** agar pencarian memiliki kesempatan menemukan solusi yang lebih baik dan tidak mudah terjebak di local optimum.

4. **Menyesuaikan parameter algoritma,** seperti suhu awal dan cooling rate pada Simulated Annealing serta ukuran populasi, laju mutasi, dan jumlah generasi pada Genetic Algorithm, untuk memperoleh hasil yang lebih baik dengan waktu pencarian yang efisien.

5. **Menguji berbagai ukuran permasalahan,** seperti jumlah kendaraan, ukuran geladak, dan kapasitas angkut, untuk mengetahui kinerja algoritma pada kondisi yang berbeda.

6. **Mengulang pengujian Genetic Algorithm** dengan 300 generasi dan mengukur waktu eksekusi seluruh algoritma pada kondisi perangkat yang sama agar hasil perbandingannya lebih adil dan konsisten.

PEMBAGIAN TUGAS

Nama	NIM	Kontribusi
M.B. Adyanti Narandita	18224026	Simulated Annealing, pengerjaan laporan, visualisasi
Derren Christopher Boenardi	18224040	Define environment, Hill Climbing, integrasi kode
Rafeyfa Dea Putri Ramadhani	18224044	Genetic Algorithm, pengerjaan laporan, run experiment

DAFTAR PUSTAKA

Atlantis Press. (n.d.). *Simulated annealing algorithm and its application*. https://www.atlantis-press.com/article/2753.pdf

Chandel, S., & Sood, M. (2019). *Searching and optimization techniques in artificial intelligence*. Semantic Scholar. https://www.semanticscholar.org/paper/Searching-and-Optimization-Techniques-in-Artificial-Chandel-Sood/5e26e100a0106a675fb6080422b166ccf18df723

Delahaye, D., Chaimatanan, S., & Mongeau, M. (2019). Simulated annealing: From basics to applications. In M. Gendreau & J.-Y. Potvin (Eds.), *Handbook of Metaheuristics* (3rd ed., pp. 1–36). Springer. https://doi.org/10.1007/978-3-319-91086-4_1

Falkenauer, E. (1996). *Genetic algorithms and grouping problems*. John Wiley & Sons.

GeeksforGeeks. (2025). *Hill climbing in artificial intelligence*. https://www.geeksforgeeks.org/artificial-intelligence/introduction-hill-climbing-artificial-intelligence/

Goldberg, D. E. (1989). *Genetic algorithms in search, optimization, and machine learning*. Addison-Wesley.

Khodra, M. L. (n.d.). *Classical vs Local Search*. https://cdn-edunex.itb.ac.id/53145-Artificial-Intelligence-Parallel-Class/194228-Beyond-Classical-Search/1693804818592_IF3170_Materi03_Seg01_BeyondClassicalSearch_LocalSearch.pdf

Khodra, M. L. (n.d.). *Genetic Algorithm*. https://cdn-edunex.itb.ac.id/53145-Artificial-Intelligence-Parallel-Class/194228-Beyond-Classical-Search/1693969141836_IF3170_Materi03_Seg05_BeyondClassicalSearch.pdf

Khodra, M. L. (n.d.). *Hill-climbing Search*. https://cdn-edunex.itb.ac.id/53145-Artificial-Intelligence-Parallel-Class/194228-Beyond-Classical-Search/1693804849395_IF3170_Materi3_Seg03_BeyondClassicalSearch_HillClimbing.pdf

Khodra, M. L. (n.d.). *Simulated Annealing*. https://cdn-edunex.itb.ac.id/53145-Artificial-Intelligence-Parallel-Class/194228-Beyond-Classical-Search/1693804872404_IF3170_Materi03_Seg04_BeyondClassicalSearch_SimulatedAnnealing.pdf

Mohammadi, A., & Sheikholeslam, F. (2023). Intelligent optimization: Literature review and state-of-the-art algorithms (1965–2022). *Engineering Applications of Artificial Intelligence*, *126*(Part D), 106959. https://doi.org/10.1016/j.engappai.2023.106959

ScienceDirect. (2024). A novel simulated annealing algorithm for large-scale optimization problems. *Artificial Intelligence in the Life Sciences*, *4*, 100009. https://doi.org/10.1016/j.ailsci.2024.100009

Topepo. (n.d.). *Feature selection using simulated annealing*. In The caret package documentation. https://topepo.github.io/caret/feature-selection-using-simulated-annealing.html

LAMPIRAN

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM	:	M.B. Adyanti Narandita/18224026 Derren Christopher Boenardi/18224040 Rafeyfa Dea Putri Ramadhani/18224044
Fakultas/Sekolah	:	STEI-K
Kode Mata Kuliah	:	IF3070
Nama Mata Kuliah	:	Dasar Intelegensi Artifisial
Judul Tugas/Proposal	:	Pengembangan Cargo Optimizer untuk Optimasi Penempatan Kendaraan pada Geladak Kapal

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

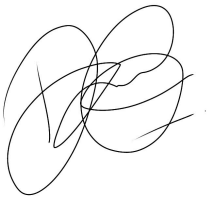
Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Sedang
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Sedang
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026



M.B. Adyanti Narandita
18224026



Derren Christopher Boenardi
18224040



Rafeyfa Dea Putri R.
18224044